

Topic 5: Data Visualization

Weeks 8, 9

Igor Razgon

igor.razgon@durham.ac.uk

Content

1. Scatter plots
2. Bar plots and line plots
3. Higher dimensional visualization

Scatter plots: detailed visualization of bi-dimensional data

Scatter plot: a basic representation of bidimensional data

- Let df be a DataFrame, $col1$ and $col2$ be two of its columns.
- Coordinates on the x -axis correspond to the values of $col1$, coordinates on the y -axis correspond to the values of $col2$.
- For each tuple (x,y) occurring in the columns, we put a point at the intersection of coordinates x and y .

Example of a scatter plot

```
In [5]: import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns
```

```
pdmarks = pd.read_csv('student_marks.csv')
pdmarks
```

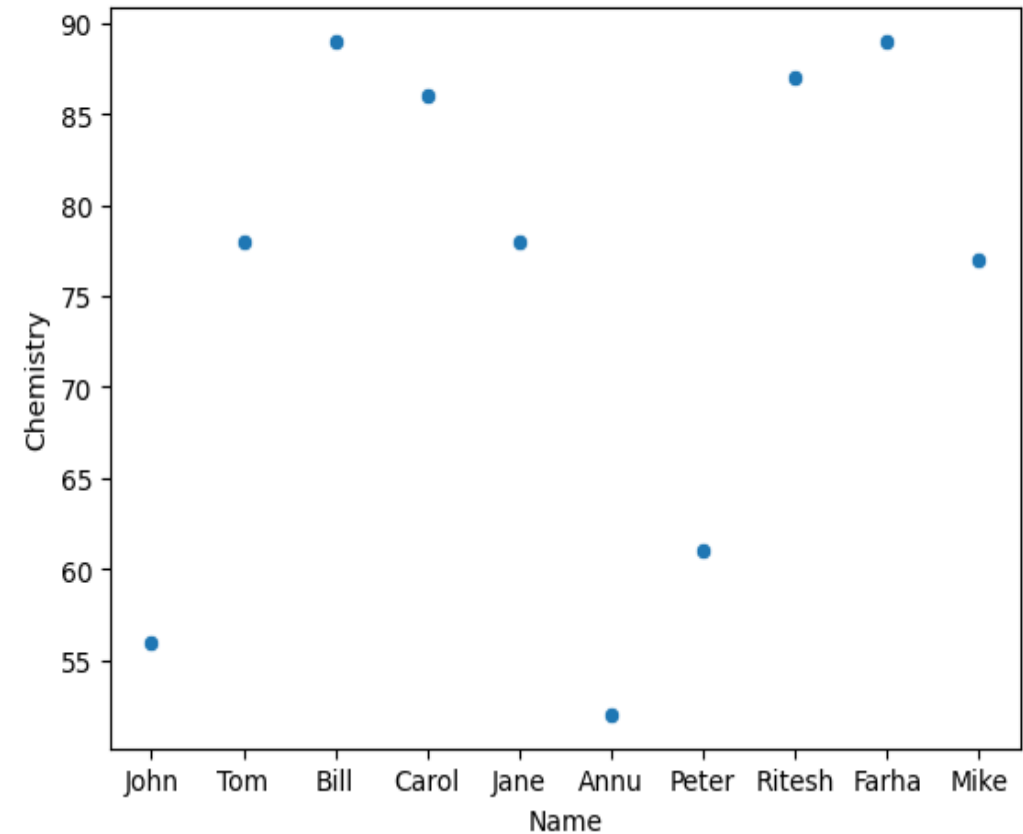
Out[5]:

	Name	Gender	DOB	Maths	Physics	Chemistry	English	Biology
0	John	M	05/04/1988	55	45	56	87	21
1	Tom	M	04/05/1987	75	96	78	64	90
2	Bill	M	25/05/1989	25	54	89	76	95
3	Carol	F	12/08/1990	78	96	86	63	54
4	Jane	F	02/09/1989	58	96	78	46	96
5	Annu	F	05/04/1988	45	87	52	89	55
6	Peter	F	04/05/1987	55	64	61	58	75
7	Ritesh	M	25/05/1989	54	76	87	56	25
8	Farha	F	12/08/1990	55	63	89	75	78
9	Mike	M	02/09/1989	96	46	77	83	58



```
In [6]: sns.scatterplot(data=pdmarks,x='Name',y='Chemistry')
```

```
Out[6]: <AxesSubplot: xlabel='Name', ylabel='Chemistry'>
```



Overlapping points

- Identical points overlap at scatter plots.
- Therefore, it is impossible to tell the *number* of tuples corresponding to a specific point in the dataset by only looking at the scatter plot.

Example of a scatter plot representing many tuples with only two points.

```
In [10]: clusterdict={      'Maths': [80,80,80,80,60,60,60,60],  
                             'Chemistry': [70,70,70,70,90,90,90,90] }  
pdcluster=pd.DataFrame(clusterdict)  
pdcluster
```

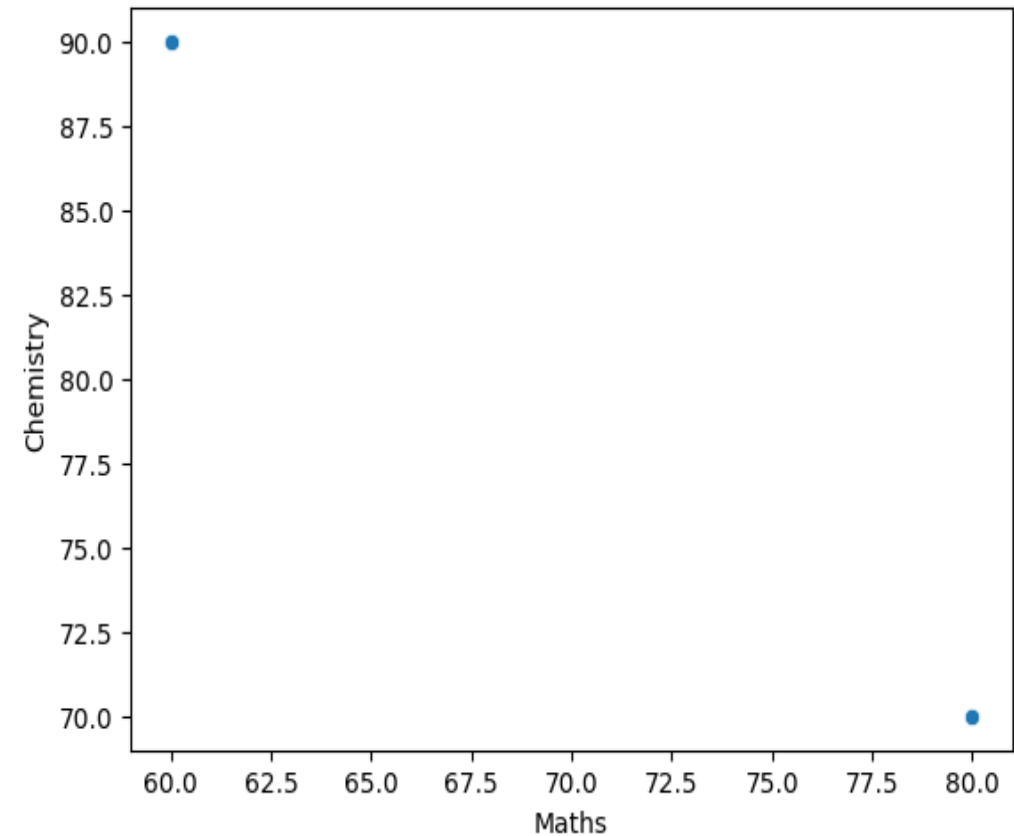
Out[10]:

	Maths	Chemistry
0	80	70
1	80	70
2	80	70
3	80	70
4	60	90
5	60	90
6	60	90
7	60	90



```
In [8]: sns.scatterplot(data=pdcluster,x='Maths',y='Chemistry')
```

Out[8]: <AxesSubplot: xlabel='Maths', ylabel='Chemistry'>



Clarifying visualization for the overlapping points

- One possibility is to present points with a random jitter effect.
- This gives an idea of presence of clustering, but the cluster size is not clearly conveyed.
- A better clarification is to use *joint plot* instead of scatter plot.
- It combines the scatter plot with a histogram for each column.

Example of a joint plot

```
In [10]: clusterdict={      'Maths': [80,80,80,80,60,60,60,60],  
                             'Chemistry': [70,70,70,70,90,90,90,90] }  
pdcluster=pd.DataFrame(clusterdict)  
pdcluster
```

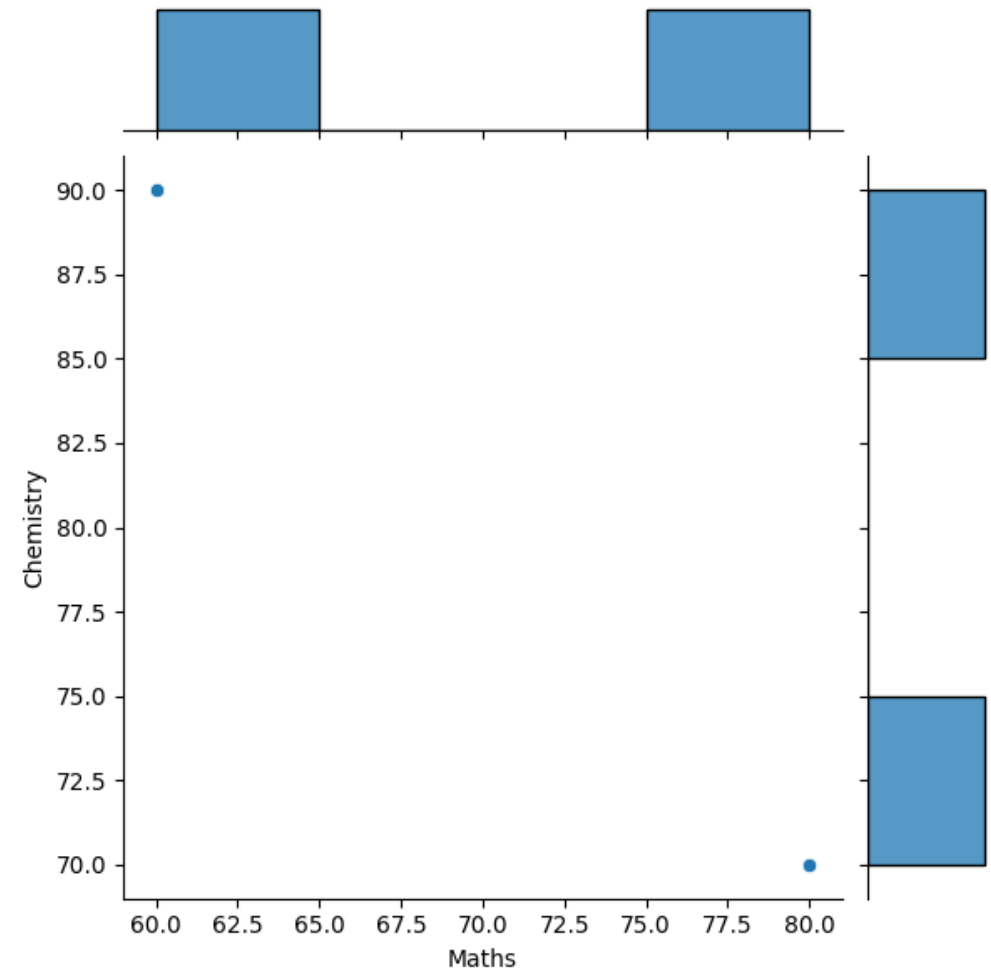
Out[10]:

	Maths	Chemistry
0	80	70
1	80	70
2	80	70
3	80	70
4	60	90
5	60	90
6	60	90
7	60	90



```
In [9]: sns.jointplot(data=pdcluster,x='Maths',y='Chemistry')
```

Out[9]: <seaborn.axisgrid.JointGrid at 0x1ce96b4f100>



Information obtained from scatter plots and joint plots

- Scatter plots visualizes all the points in a bi-dimensional distribution.
- We can tell whether the points are uniformly distributed along a large region or there are several small clusters that are distant from each other.
- Joint plots provide additional information about clustering of overlapping points
- It is better to use bar plots or line plots for visualization of aggregate statistics such as means.

Bar plots and line plots: aggregate visualization of bi-dimensional data

Bar plots: generic view

- Like scatter plots, bar plots visualize dependencies of two columns.
- That is, we consider columns *col1* and *col2* of a DataFrame *df*.
- Values of *col1* occur along the *x*-axis, values of *col2* occur along the *y*-axis.
- Each value of *col1* is associated with a rectangle and an error bar (their interpretation is discussed in the next slide).

Bar plots: interpretation of rectangles and error bars

- The height of each rectangle represents the aggregate of a statistical measure of values of *col2* for the given value of *col1*. (We use the mean for this purpose.)
- The length of the error bar represents the deviation of the actual set of values from the mean.
- By default, the length is 0.95 of the confidence interval.
- That is, the bar starts at $y = \text{mean} - 1.96 * \sigma / n$ and ends at $y = \text{mean} + 1.96 * \sigma / n$, where
 - *mean* is the mean of values *col2* for the value of *col1*,
 - *n* is the amount of such values,
 - σ is their standard deviation

Example of a barplot

```
In [1]: import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

testdict={
    'key1': ['a','b','a','a','a','b','a','b','b'],
    'key2': ['c','c','c','c','d','d','d','d','c'],
    'num1': [1,1,1,1,4,4,6,7,8],
    'num2': [40,35,30,100,25,20,15,10,5]
}

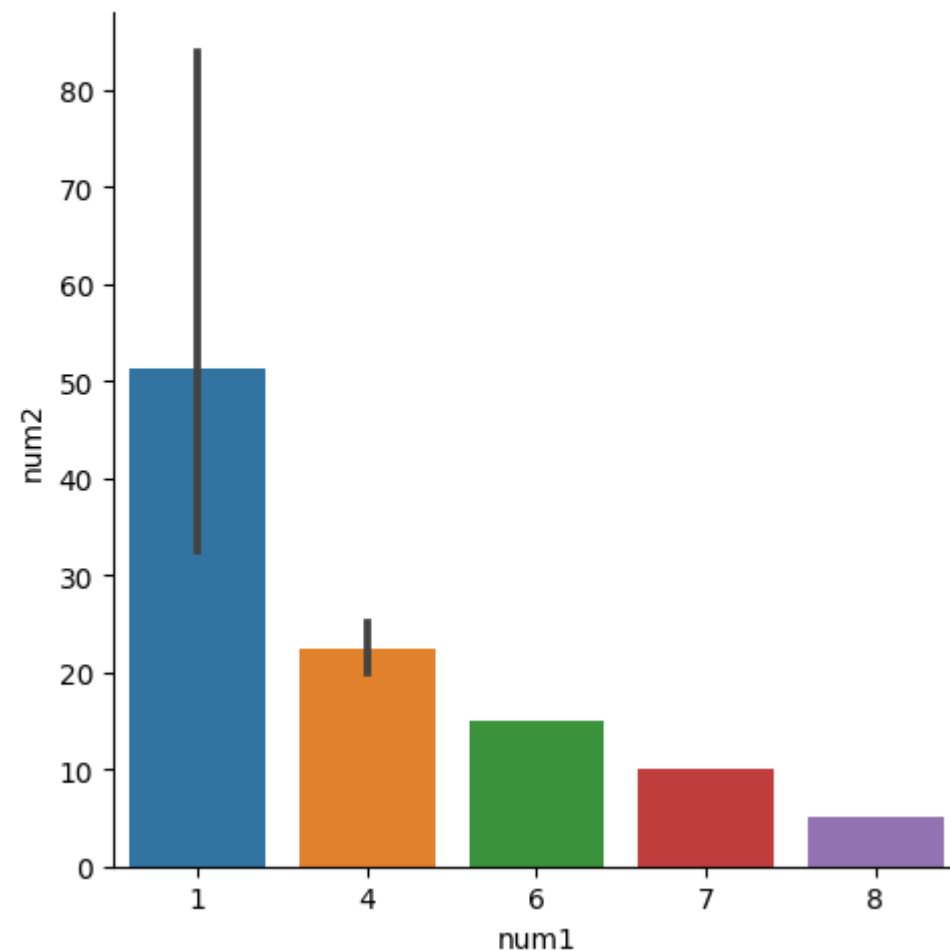
pdtest=pd.DataFrame(testdict)
pdtest
```

Out[1]:

	key1	key2	num1	num2
0	a	c	1	40
1	b	c	1	35
2	a	c	1	30
3	a	c	1	100
4	a	d	4	25
5	b	d	4	20
6	a	d	6	15
7	b	d	7	10
8	b	c	8	5

```
In [2]: sns.catplot(data=pdtest,x="num1",y="num2",kind='bar')
```

Out[2]: <seaborn.axisgrid.FacetGrid at 0x2cdebeebc10>



What information can be obtained from bar plots

- The rectangles specify the *average* values of *col2* for each value of *col1*.
- The error bars convey the discrepancy.
 - If the bar is long, this means that the values can be far away from the average.
 - If the bar is short, then the values are concentrated around the average.

Limitation: bar plots are not self-evident

- The meaning of rectangles and error bars needs to be explained.
- This is especially so as they can be easily confused with *histograms*.

Line plots

- The x and y axes have the same role as for bar plots
- Three steps construction for a line plot
 1. Specify the points of averages of averages of $col2$ for each value of $col1$.
 2. Connect the dots with a curve (through interpolation).
 3. Add shaded regions around the points to demonstrate discrepancy.

Example of a line plot

```
In [3]: linetest={'x': [1,1,1,2,2,2,2,3,3,3,3,4,5,6],  
                'y': [10,12,140,15,15.5,15.7,20,20.5,20.8,22,23,40,80,160]}  
pdline=pd.DataFrame(linetest)  
pdline
```

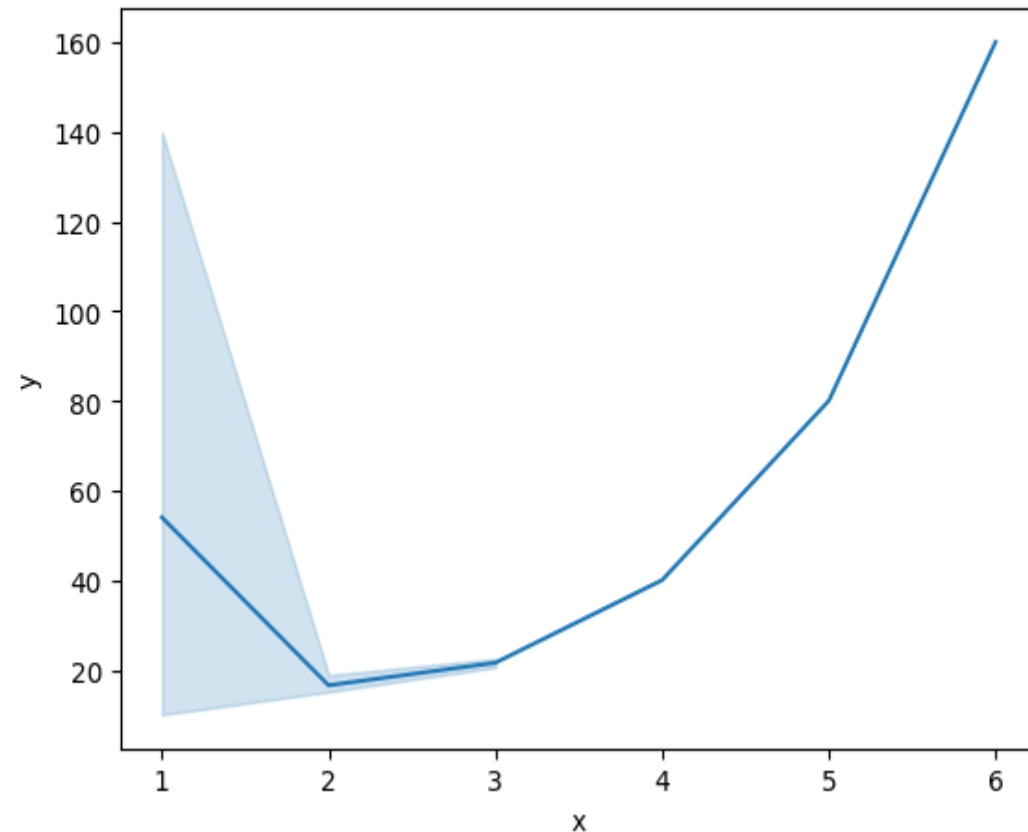
Out[3]:

	x	y
0	1	10.0
1	1	12.0
2	1	140.0
3	2	15.0
4	2	15.5
5	2	15.7
6	2	20.0
7	3	20.5
8	3	20.8
9	3	22.0
10	3	23.0
11	4	40.0
12	5	80.0
13	6	160.0



```
In [4]: sns.lineplot(x='x', y='y', data=pdline)
```

Out[4]: <AxesSubplot: xlabel='x', ylabel='y'>



Line plots vs bar plots

- Line plots carry the same factual information as bar plots: aggregate statistics and discrepancies.
- However, the line plots also visualize interpolation so the analyst can clearly see functional dependencies between columns (as suggested by the factual data).
- Also, curves may have a clearer meaning for 'lay viewers' than rectangles and error bars.

Higher dimensional visualization

Boosting dimension by the *hue* parameter

- Available for each of the plots considered so far
- Let us consider the application for bar plots

Example of a bar plot with several colours used

```
In [1]: import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import seaborn as sns

testdict={'key1': ['a', 'b', 'a', 'a', 'a', 'b', 'a', 'b', 'b'],
          'key2': ['c', 'c', 'c', 'c', 'd', 'd', 'd', 'd', 'c'],
          'num1': [1, 1, 1, 1, 4, 4, 6, 7, 8],
          'num2': [40, 35, 30, 100, 25, 20, 15, 10, 5] }

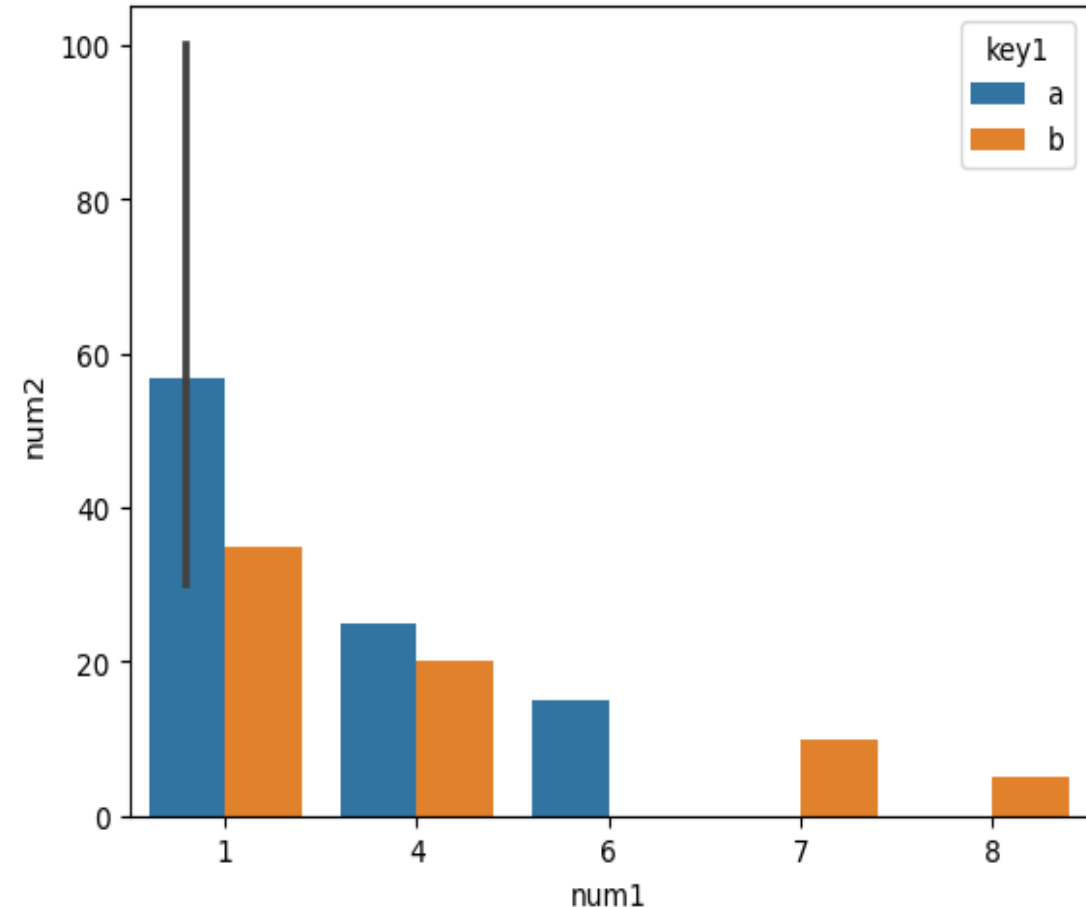
pdtest=pd.DataFrame(testdict)
pdtest
```

Out[1]:

	key1	key2	num1	num2
0	a	c	1	40
1	b	c	1	35
2	a	c	1	30
3	a	c	1	100
4	a	d	4	25
5	b	d	4	20
6	a	d	6	15
7	b	d	7	10
8	b	c	8	5

```
In [2]: sns.barplot(data=pdtest,x="num1",y="num2",hue='key1')
```

Out[2]: <AxesSubplot: xlabel='num1', ylabel='num2'>



Pairplot: all pairs of columns visualized on the same diagram

- Consider a data frame df with columns $1, \dots, n$.
- A pairplot can be seen as a $n \times n$ table with the table entry at row i can column j correspond to diagram visualizing columns i and j of df .
- The diagonal entries (same row and column) of the table are histograms for the corresponding columns of df .
- The remaining entries df are scatterplots for the corresponding pairs of columns of df .

Example of a pair plot

```
In [3]: pdmarks = pd.read_csv('student_marks.csv')  
mfc=pdmarks[['Maths','Physics','Chemistry']]  
mfc
```

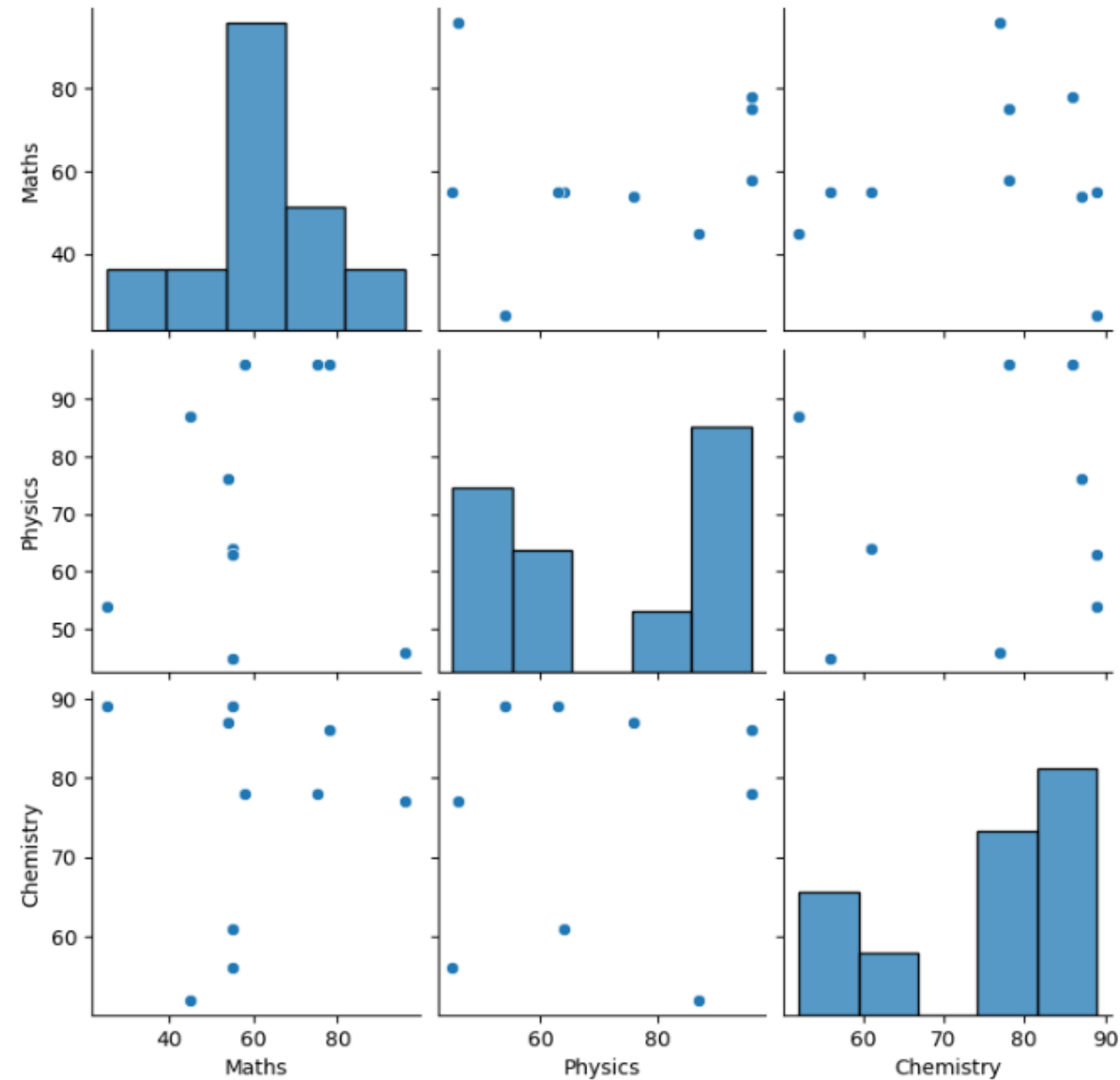
Out[3]:

	Maths	Physics	Chemistry
0	55	45	56
1	75	96	78
2	25	54	89
3	78	96	86
4	58	96	78
5	45	87	52
6	55	64	61
7	54	76	87
8	55	63	89
9	96	46	77



```
In [4]: sns.pairplot(data=mfc)
```

```
Out[4]: <seaborn.axisgrid.PairGrid at 0x283d2299930>
```



Limitations of pairplots

- Not suitable for too many columns (visualizing more than 5 columns this way would lose the clarity).
- Dependencies between all pairs of columns do not necessarily convey dependencies between their triples and quadruples.

Heatmap: visualizing the whole data frame

- DataFrame **with numerical entries** can be visualized as a whole using heatmaps.
- As for the standard print operation, the rows correspond to the index and the columns are just DataFrame columns
- But the numbers are replaced by colours, and different numbers correspond to different colour intensity.

Example of a heat map

```
In [5]: testheat={'key1': [5,8,8,7,7,9,9,9],  
                  'key2': [1,4,3,3,45,2,2,2],  
                  'num1': [1,2,3,4,5,6,7,8],  
                  'num2': [40,35,30,25,20,15,10,5]}  
  
pdheat=pd.DataFrame(testheat)  
pdheat
```

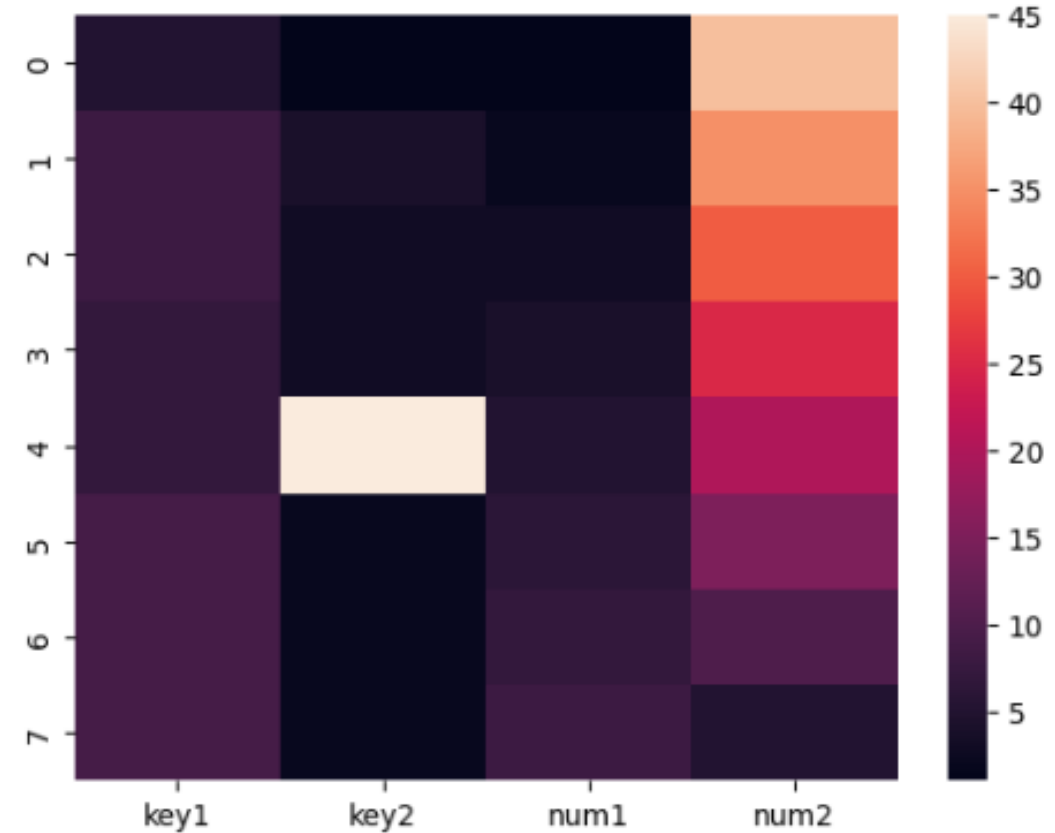
Out[5]:

	key1	key2	num1	num2
0	5	1	1	40
1	8	4	2	35
2	8	3	3	30
3	7	3	4	25
4	7	45	5	20
5	9	2	6	15
6	9	2	7	10
7	9	2	8	5



```
In [6]: sns.heatmap(pdheat)
```

Out[6]: <AxesSubplot: >



Summary

- We have considered the following visualization approaches.
 - Histograms visualizing one-dimensional data.
 - Scatter plots providing *detailed* visualization of bi-dimensional data
 - Bar plots and line plots providing *aggregate* visualization of bi-dimensional data
 - Several approaches to visualization of data of a higher dimension
- There are plenty of other visualization (plenty of data visualization books can be found by googling).
- The ultimate goal of data visualization is **story-telling** that we will consider during the next class.